

Introduction to Unix systems

Lecture 03

Computer Physics

22 October 2026

Data streams, pipelines and text processing

Jacek Gatlik

email: jacek.gatlik@agh.edu.pl
www: https://galaxy.agh.edu.pl/~jgatlik/

Faculty of Physics and Applied Computer Science,
AGH University of Krakow

Outline

Standard input, output and error streams

Regular expressions

Finding files and directories

Translating and deleting characters with tr

Searching text with grep

Transforming text with sed

Processing records and calculations with awk

Section 1

Standard input, output and error streams

Three standard streams

A command normally inherits three open streams from the shell. Each has a conventional **file descriptor number**:

Stream	Name	Number	Purpose
Standard input	`stdin`	`0`	Data read by the command.
Standard output	`stdout`	`1`	Normal results.
Standard error	`stderr`	`2`	Diagnostics and warnings.

- In an interactive terminal, stdin usually receives keyboard input. Both stdout and stderr usually appear in the terminal.
- The shell can connect these streams to files or other commands.

Results and diagnostics have different roles

A simulation can write numerical results to stdout and progress messages to stderr. Keeping them separate helps preserve clean data.

Reading from standard input

Reuse `data/motion.txt` from Lecture 01: one header followed by five measurement rows.

```
wc -l data/motion.txt
wc -l < data/motion.txt
```

The corresponding outputs are:

```
6 data/motion.txt
6
```

- First command: `wc` receives a filename argument and opens the file itself.
- Second command: the shell opens the file and connects it to **stdin**. With no filename, `wc` reads that stream.
- `wc -l` counts newline characters. Here, each of the six lines ends with a newline.

Redirection belongs to the shell

`<` redirects standard input, descriptor `0`. The operator and its filename are not arguments passed to `wc`.

Saving and appending standard output

```
head -n 3 data/motion.txt > preview.txt
echo "Preview created" > session.log
echo "Preview checked" >> session.log
```

- `>` sends stdout to a file. With normal Bash settings, it creates the file if absent or truncates an existing file to zero length.
- `>>` appends stdout to the end of a file. It also creates the file if absent.
- `preview.txt` contains the header and two data rows. `session.log` contains both messages.
- `>` is short for `1>`. These operators leave stderr at its current destination.

The shell opens the output file before the command runs

Using the same file as input and a `>` destination destroys its original contents, even if the command later fails. Write to a different file when processing data.

Separating results from diagnostics

Assume `data/motion.txt` exists and `missing.txt` does not:

```
cat data/motion.txt missing.txt > copy.txt 2> errors.log
```

- `copy.txt` receives stdout: the contents of the existing file.
- `errors.log` receives stderr: the diagnostic about `missing.txt`. Neither stream appears in the terminal here.
- `2>` replaces the error log. Use `2>>` to append diagnostics instead.

To discard diagnostics deliberately:

```
cat data/motion.txt missing.txt > copy.txt 2> /dev/null
```

Discarding a message does not fix the problem

`/dev/null` discards anything written to it. The missing file still causes `cat` to report failure through its exit status.

Combining streams: order matters

Both streams go to the same log:

```
cat data/motion.txt missing.txt > run.log 2>&1
```

1. `> run.log` connects stdout to the file.
2. `2>&1` makes stderr use stdout's current destination: now `run.log`.

Changing the order changes the result:

```
cat data/motion.txt missing.txt 2>&1 > run.log
```

1. Stderr first takes stdout's current destination, normally the terminal.
2. Only stdout then moves to `run.log`. Diagnostics still appear in the terminal.

The shell processes redirections from left to right

`2>&1` duplicates descriptor `1`'s destination at that moment. It does not follow later changes to stdout.

Connecting commands with a pipeline

The pipe operator, `|`, connects one command's **stdout** to the next command's **stdin**.

```
tail -n +2 data/motion.txt | wc -l
```

- `tail -n +2` prints from line 2 onward, omitting the header.
- `wc -l` reads the stream and prints `5`: the number of measurement rows.
- Commands can run concurrently. The pipe transfers data without an intermediate disk file.

Diagnostics can go to a separate file while data flows through the pipe:

```
cat data/motion.txt missing.txt 2> errors.log | wc -l
```

This prints `6`, including the header. The missing-file diagnostic goes to `errors.log`.

An ordinary pipe carries stdout only

Stderr keeps its current destination unless you redirect it. `command 2>&1 | next` sends both streams into the pipe.

Saving intermediate data with tee

`tee` reads stdin and copies it to a file **and** to stdout, so the pipeline can continue.

```
tail -n +2 data/motion.txt | tee values.txt | wc -l > count.txt
```

Stage	Result
`tail -n +2`	Sends five measurement rows into the pipeline.
`tee values.txt`	Saves those rows and passes the same data onward.
`wc -l`	Writes the count, `5`, to `count.txt`.

`tee` normally overwrites its output file. Use `tee -a values.txt` to append instead.

A saved copy and a continuing stream

`values.txt` keeps the intermediate data for inspection. `count.txt` keeps the final result. The original `data/motion.txt` remains unchanged.

Predict: what changes if you remove `> count.txt`?

Regular expressions: matching text

A **regular expression**, or **regex**, is a pattern that describes which text should match. Ordinary characters match themselves. Special characters express more general rules.
The simplest regex is a literal sequence such as `cat`:

Pattern	Input text	Result
`cat`	`the cat sleeps`	Matches `cat`.
`cat`	`concatenate`	Matches `cat` inside the word.
`cat`	`Cat`	No match with case-sensitive matching.

- A search tries possible starting positions. The whole pattern must fit, but it may match only part of a line.
- Adjacent pattern elements match adjacent text, in order. For example, `cat` requires `c`, then `a`, then `t`.

The pattern describes a match

The tool uses the match: `grep` selects matching lines. `sed` can replace matching text.

Regular expressions: characters and positions

Character patterns match **one character**. Anchors mark positions without matching a character.

Pattern	Meaning when matching a line of text
`.`	Any single character, including a space.
`[abc]`	One of the listed characters: `a`, `b` or `c`.
`[0-9]`	One digit, from `0` to `9`.
`[^0-9]`	One character other than a digit.
`^` and `$`	Start and end of the line, respectively.
`[.]` or `\.`	A literal dot. The backslash escapes its special meaning.

Example: `^run[0-9]$` requires a line containing exactly `run` followed by one digit. It matches `run2`, but rejects `rerun2` and `run20`.

The position of a special character matters

At the start of a pattern, `^` anchors the match. Immediately after `[`, it negates the character set.

Regular expressions: repetition and alternatives

In **extended regular expressions (ERE)**, repetition applies to the **preceding element**: a character, a bracket expression or a parenthesised group.

Form (ERE)	Meaning	Example matches
`a*`	Zero or more repetitions.	Empty text, `a`, `aaa`.
`a+`	One or more repetitions.	`a`, `aa`, `aaa`.
`a?`	Zero or one occurrence.	Empty text or `a`.
`a{3}`	Exactly three repetitions.	`aaa`.
`a{2,4}`	Two to four repetitions.	`aa`, `aaa`, `aaaa`.
`(cat|dog)`	Either alternative.	`cat` or `dog`.

`ab+` repeats only `b`. `(ab)+` repeats the whole group, matching `ab`, `abab`, etc.

Reading a complete pattern

`^run-[0-9]+[.]dat$` means: start of line, literal `run-`, one or more digits, literal `.dat`, end of line.

It matches `run-2.dat` and `run-12.dat`. It rejects `run-.dat` and `run-2Xdat`.

Regular expressions and filename patterns

The shell and `find -name` use **filename patterns** (globs). They match complete names and use different operators from regular expressions:

Intended match	Filename pattern	ERE
Any sequence of characters	`*`	`.*`
One character	`?`	`.`
A literal dot	`.`	`[.]`
A complete name ending in `.dat`	`*.dat`	`^.*[.]dat$`

In a glob, `run-?.dat` accepts `run-2.dat` and `run-x.dat`, but rejects `run-12.dat`. The `?` stands for exactly one character.

Quotes let the receiving tool interpret the pattern

The shell expands unquoted globs before running a command. In `find . -name '*.dat'`, quotes let `find` receive the pattern and apply it during its search.

Section 3

Finding files and directories

Finding files by name and type

`find` searches a directory tree recursively. By default, it prints the paths of matching entries to stdout.

```
find . -type f -name '*.dat'
find . -type d -name 'run-*'
find data -type f -iname '*.txt'
```

- The starting path determines where the search begins: `.` means the current directory, while `data` limits the search to that subtree.
- `-type f` selects regular files. `-type d` selects directories.
- `-name` matches the entry's name without its parent directories. Matching is case-sensitive.
- `-iname` ignores case: `'*.txt'` also matches names ending in `.TXT`.

The whole entry name must match

`'*.dat'` accepts `run-2.dat`, but rejects `run-2.dat.old`. `'run-*'` selects names beginning with `run-`.

Combining search conditions

Select regular files whose names end in `.dat` or `.txt`:

```
1 find . -type f \( -name '*.dat' -o -name '*.txt' \)
```

- Adjacent tests mean **AND**: all conditions must match.
- `-o` means **OR**. Parentheses group the filename alternatives, so `-type f` applies to both.
- Escape the parentheses as `\(` and `\)` so the shell passes them to `find`.

Select large files modified recently:

```
1 find . -type f -size +100M -mtime -1
```

Modification time uses elapsed 24-hour periods

`-mtime -1` selects files changed in the preceding 24 hours, rather than only the current calendar day. `-size +100M` selects files larger than 100 MiB. In GNU `find`, $1\,\mathrm{MiB} = 1024^2$ bytes.

Running commands on matching files

First inspect the selected paths:

```
find data -type f -name '*.txt' -print
```

Two alternatives for counting lines in the matching files:

```
find data -type f -name '*.txt' -exec wc -l -- {} +
find data -type f -name '*.txt' -exec wc -l -- {} \;
```

- `-exec` runs a command on the matching files.
- `{}` marks where paths go. `+` passes several paths to each `wc` invocation.
- `\;` runs `wc` once per file. The backslash protects the semicolon from the shell.
- `--` ends the option list for `wc`. The following arguments are filenames.

Filenames remain separate arguments

Each path is a separate argument, including names containing spaces. Before a modifying action, reuse the same search tests with `-print` to inspect the selected paths.

tr: command syntax and options

`tr` translates or deletes individual characters and can squeeze repeated characters.

```
tr [options] 'set1' ['set2']
```

- Input comes from stdin; output goes to stdout. `tr` takes no input filename argument.
- With two sets and no options, characters in `set1` become the corresponding characters in `set2`.
- Square brackets mark optional parts of this syntax guide. Do not type them.

Option	Meaning
`-d`	Delete characters listed in `set1`.
`-s`	Squeeze adjacent repeats of a character in the last set.
`-c`	Use the complement of `set1`: all characters outside that set.

Reading a file through standard input

`tr 'abc' 'XYZ' < input.txt > output.txt` saves the transformed stream in a different file.

Example: translating characters

Map characters by position, or change the case of ASCII letters:

```
printf '%s\n' 'cab 203' | tr 'abc' 'XYZ'
printf '%s\n' 'Run 2: warn' | tr '[:lower:]' '[:upper:]'
```

Outputs, in the same order:

```
ZXY 203
RUN 2: WARN
```

- The first command maps `a` to `X`, `b` to `Y` and `c` to `Z`. Other characters stay unchanged.
- `[:lower:]` and `[:upper:]` are character classes. Reversing them converts uppercase letters to lowercase.

Translation works character by character

The characters in `'abc'` need not occur next to each other. `tr` applies each mapping wherever that character occurs.

Example: deleting and retaining characters

Remove digits from a name, then extract digits from a short record:

```
printf '%s\n' 'run12.dat' | tr -d '[:digit:]'
printf '%s\n' 'run12 step34' | tr -cd '[:digit:]\n'
```

Outputs, in the same order:

```
run.dat
1234
```

- The first command removes `1` and `2`.
- `-cd` keeps only digits and newline characters. `\n` preserves the end of the output line.
- Deleting letters and spaces joins the remaining digits: no separator is inserted between `12` and `34`.

Character sets have their own notation

`tr` does not interpret regular expressions. Write the digit class as `[:digit:]`, without the extra outer brackets used in a regex.

Example: squeezing repeated characters

Collapse repeated spaces, or put each status on a separate line:

```
printf '%s\n' 'OK   WARN    FAIL' | tr -s ' '
printf '%s\n' 'OK   WARN    FAIL' | tr -s '[:blank:]' '\n'
```

Outputs, in the same order:

```
OK WARN FAIL
OK
WARN
FAIL
```

- The first command reduces each run of spaces to one space.
- GNU `tr` repeats the last character of a shorter `set2`. Here, spaces and tabs therefore become newlines; `-s` then squeezes repeated newlines.

Spaces and tabs differ from line breaks

`[:blank:]` contains spaces and tabs. `[:space:]` also includes newline characters.

Section 5

Searching text with grep

grep: command syntax

`grep` selects input lines that match a pattern. By default, it prints each selected line in full.

```
grep [options] 'pattern' [file...]
command | grep [options] 'pattern'
```

- `pattern` describes the text to find. Single quotes protect it from interpretation by the shell.
- `file...` means one or more input files. With no file argument, `grep` reads standard input.
- Square brackets mark optional parts in this syntax guide; do not type them. Options normally come before the pattern.
- Selected lines go to standard output. Searching does not change the input files.

Searching several files

When several files are searched, matching lines normally include their source filename.

grep: matching options

Choose how the pattern is interpreted and which lines are selected:

Option	Meaning
Default	Use basic regular expressions (BRE); match case exactly.
`-E`	Use extended regular expressions (ERE), including `+`, `?`, `(...)` and `|`.
`-F`	Treat patterns as literal text (fixed strings).
`-i`	Ignore differences between uppercase and lowercase.
`-v`	Select lines that do not match any supplied pattern.
`-x`	Require the pattern to match the whole line.

Choose one pattern mode

Use the default BRE mode, `-E` or `-F`. Other options can be combined: `-Ei` means `-E -i`, and `-Ev` means `-E -v`.

grep: output and input options

Control the result format and search several patterns or files:

Option	Meaning
`-n`	Prefix each selected line with its input line number.
`-c`	Print the number of selected lines for each input file.
`-l`	Print only the names of files containing selected lines.
`-r`	Search files inside a directory and its subdirectories.
`-e 'pattern'`	Supply a pattern; repeat `-e` to select lines matching any of the patterns.
`-f file`	Read patterns from `file`, one per line.

Lines and matches are different

`-c` counts selected lines, not individual occurrences. A line containing the pattern three times still counts as one line.

One dataset for the following examples

Create `data/energy.txt` with one comment header and six illustrative simulation records:

```
# step time_s energy status
0 0.0 10.0 OK
1 0.1 9.8 OK
2 0.2 15.2 WARN
3 0.3 9.7 OK
4 0.4 18.4 FAIL
5 0.5 9.9 OK
```

```
mkdir -p data
nano data/energy.txt
export LC_ALL=C
```

- Separate fields with spaces and end every line with a newline.
- Run subsequent commands from the directory containing `data/`.
- `LC_ALL=C` gives predictable character matching and a dot as the decimal separator in calculations.

Example: locating a status record

Find a warning, locate its input line and search for a literal time:

```
grep 'WARN' data/energy.txt
grep -n -i 'warn' data/energy.txt
grep -F '0.2' data/energy.txt
```

Outputs, in the same order:

```
2 0.2 15.2 WARN
4:2 0.2 15.2 WARN
2 0.2 15.2 WARN
```

- The second search finds uppercase `WARN` even though the supplied pattern is lowercase.
- In the third search, the dot in `0.2` is an ordinary character because `-F` is used.

Input line numbers are not simulation steps

The warning is on input line 4, at simulation step 2. The header is also counted as an input line.

Example: selecting complete step numbers

Select only the records whose first field is step 2 or step 4:

```
grep -E '^[24] ' data/energy.txt
```

```
2 0.2 15.2 WARN
4 0.4 18.4 FAIL
```

- The match starts at the beginning of the record, so digits in the time or energy fields do not qualify.
- The bracket expression allows either of the two requested steps.
- The space after `[24]` ends the step field: this pattern does not select step `20` or `40`.

Include field boundaries in a text search

A search for `'2'` alone would also find records containing that digit elsewhere on the line.

Example: allowing spaces and tabs

Allow flexible spacing between fields and before a comment:

```
grep -E '^2[[:space:]]+0[.]2[[:space:]]' data/energy.txt
grep -E '^[[:space:]]*#' data/energy.txt
```

Outputs, in the same order:

```
2 0.2 15.2 WARN
# step time_s energy status
```

- `[[:space:]]` matches a whitespace character, such as a space or tab.
- The first pattern selects step 2 at time `0.2`; the separator may contain several spaces or tabs.
- The second pattern finds a comment even if its `#` is indented with whitespace.

A decimal point is part of the data

`0[.]2` requires the literal text `0.2`. The following whitespace ends the time field, so `0.20` does not qualify.

Example: selecting exceptional statuses

Find records ending with either `WARN` or `FAIL`:

```
grep -E '[[:space:]](WARN|FAIL)$' data/energy.txt
grep -e ' WARN$' -e ' FAIL$' data/energy.txt
```

Both commands produce:

```
2 0.2 15.2 WARN
4 0.4 18.4 FAIL
```

- The first command uses one ERE with two status alternatives.
- The second supplies two separate patterns, using the single-space separators in our dataset.
- The final `$` rejects a longer status such as `WARNING` or `FAILURE`.

Read the vertical bar in context

Here, the quoted `|` is part of the search pattern. In `command | grep 'WARN'`, it connects two commands.

Example: saving records and counting exceptions

Save data records, count them, then count exceptional statuses:

```
grep -Ev '^[[:space:]]*(#|$)' data/energy.txt > measurements.txt
wc -l < measurements.txt
grep -Ec '[[:space:]](WARN|FAIL)$' data/energy.txt
```

Outputs from the two counting commands:

```
6
2
```

- The first pattern recognises comments and blank lines, allowing leading whitespace. Inverting it leaves the data records.
- `measurements.txt` contains six records and no header; the original dataset still contains seven lines.
- Two records have an exceptional status: step 2 has `WARN`, and step 4 has `FAIL`.

Section 6

Transforming text with sed

sed: command syntax

`sed` is a **stream editor**: its script describes how to transform the input text.

```
sed [options] 'script' [file...]
```

By default, it reads one line, applies the script, prints the result, then repeats for the next line. With no file argument, it reads stdin.

Option	Meaning
`-n`	Disable automatic printing.
`-E`	Use extended regular expressions (ERE).
`-e 'script'`	Add a script; repeat this option to combine scripts.
`-f file`	Read the editing script from a file.

Output is a new stream

Ordinary `sed` commands write to stdout. They leave the input file unchanged.

sed: addresses and editing commands

Each editing command can have a line selector called an **address**:

```
[address]command
```

Address	Selected lines
Omitted	Every input line.
`3`	Physical line 3; counting starts at 1.
`2,4`	Lines 2 through 4, including both endpoints.
`/pattern/`	Lines matching a regular expression.
`$`	The last input line.

`p` prints the selected line. `d` discards it and immediately moves to the next input line.

Printing selected lines

Use `-n` with `p` to output only the selected lines. Without `-n`, `p` adds a second copy to the automatic output.

sed: substitution syntax

The `s` command replaces matching text within a selected line:

```
[address]s/pattern/replacement/flags
```

- `pattern` is a regular expression: BRE by default, or ERE with `-E`.
- `replacement` supplies the new text. Inside it, `&` inserts the entire matched text.

Flags	Replacement within each selected line
None	Replace the first match.
`g`	Replace every match.
`2`	Replace only the second match.

Combining editing commands

A semicolon separates commands in one script. Commands run from left to right on the current line; after `d`, later commands do not process that line.

Example: replacing repeated text

Compare two edits of a line containing two warnings:

```
printf '%s\n' 'WARN WARN OK' | sed 's/WARN/CHECK/'
printf '%s\n' 'WARN WARN OK' | sed 's/WARN/CHECK/g'
```

Outputs, in the same order:

```
CHECK WARN OK
CHECK CHECK OK
```

- The first result still contains one `WARN`.
- The second result changes both warnings to `CHECK`.
- In both results, `OK` stays unchanged because it does not match the pattern.

A short test line reveals the difference

`sed 's/WARN/CHECK/' data/energy.txt` changes the only warning in our dataset. Two warnings on the test line make the effect of `g` visible.

Example: editing selected lines

Change one status by position, then label warning records by content:

```
sed '3s/OK/PASS/' data/energy.txt
sed '/WARN/s/^/CHECK: /' data/energy.txt
```

Changed lines, in command order:

```
1 0.1 9.8 PASS
CHECK: 2 0.2 15.2 WARN
```

- Physical line 3 is the measurement at step `1`: the comment header is also counted.
- The warning record keeps its fields and receives the prefix `CHECK:` at the beginning of the line.

These outputs show excerpts

Both commands also print all other lines unchanged. A content-based selection still finds the warning if its position in the file changes.

Example: printing or removing selected lines

Extract the first three measurements, after the comment header:

```
sed -n '2,4p' data/energy.txt
```

```
0 0.0 10.0 OK
1 0.1 9.8 OK
2 0.2 15.2 WARN
```

Select a warning record, or remove the header from the output:

```
sed -n '/WARN/p' data/energy.txt
sed '/^#/d' data/energy.txt
```

- The first command outputs only the measurement at step `2` with status `WARN`.
- The second outputs all six measurements, starting with `0 0.0 10.0 OK`. Only the line beginning with `#` is discarded.

Example: preparing comma-separated data

Our fields contain no spaces or commas. Remove the comment header and convert the field separators to commas:

```
sed -E '/^#/d; s/[[:space:]]+/,/g' data/energy.txt > energy.csv
cat energy.csv
```

```
0,0.0,10.0,OK
1,0.1,9.8,OK
2,0.2,15.2,WARN
3,0.3,9.7,OK
4,0.4,18.4,FAIL
5,0.5,9.9,OK
```

- The header is discarded before substitution.
- Each run of spaces or tabs between fields becomes one comma; field values and record order are preserved.
- `energy.csv` receives the six converted records. `data/energy.txt` is unchanged.

Section 7

Processing records and calculations with awk

awk: command syntax and rules

`awk` selects records, works with their fields and calculates results:

```
awk [options] 'pattern { action }' [file...]
```

The quoted program contains one or more rules. A pattern selects records; an action says what to do with them. With no input file, `awk` reads stdin.

Normal input rule	Meaning
`pattern { action }`	Run the action when the pattern is true.
`{ action }`	Run the action for every input record.
`pattern`	Print each matching record: the default action is `print $0`.

A pattern can be a regular expression such as `/WARN/` or a condition such as `$3 > 12`. Regular expressions use ERE syntax.

Protect the program from the shell

Single quotes keep `$1`, spaces and braces inside the program passed to `awk`.

awk: records and fields

By default, each input line is one **record**. `awk` splits it into **fields** before checking the rules.

Variable	Meaning
`$0`	The complete current record.
`$1`, `$2`, …	First field, second field, and so on.
`NF`	Number of fields in the current record.
`NR`	Records read so far, across all input files.
`FS`	Input field separator. By default, runs of spaces or tabs separate fields.
`OFS`	Output field separator, a single space by default.

print builds an output record

`print $1, $3` writes two values separated by `OFS`, followed by a newline. `print` on its own writes the complete record.

awk: options and execution order

Option	Purpose
`-F separator`	Set `FS`, for example `-F,` for comma-separated fields.
`-v name=value`	Set a variable before the program starts, for example `-v limit=12`.
`-f program.awk`	Read the program from a file instead of an inline quoted argument.

Execution order:

1. `BEGIN { action }` runs once, before reading input. Use it for setup or a report header.
2. For each record, `awk` splits the fields, checks normal rules in their written order and runs every matching action.
3. `END { action }` runs once, after all input. Use it for totals or a final summary.

`BEGIN` and `END` rules are optional; when present, each needs an action in braces.

Example: inspecting a measurement record

Inspect the first data row in `data/energy.txt`:

```
awk 'NR == 2 {print $0; print $1, $3; print NR, NF}' data/energy.txt
```

```
0 0.0 10.0 OK
0 10.0
2 4
```

- `NR == 2` selects the second input line: the header occupies line 1, so this is step `0`.
- The first output line reproduces the measurement.
- The second line keeps only the step and its energy, `10.0`.
- The last line confirms that input line `2` contains `4` fields.

The semicolons separate three statements in the same action.

Example: selecting records with conditions

Select measurements with energy greater than 12:

```
awk '!/^#/ && $3 > 12 {print $1, $3, $4}' data/energy.txt
```

```
2 15.2 WARN
4 18.4 FAIL
```

- `!` `/^#/` rejects the comment header. `!` means “not” and `&&` requires both conditions to be true.
- `$3 > 12` compares the energy field with a number; the output keeps the step, energy and status.

Select successful measurements by their status instead:

```
awk '!/^#/ && $4 == "OK" {print $1, $3}' data/energy.txt
```

This prints steps `0`, `1`, `3` and `5` with their energies. `==` tests equality; `=` assigns a value.

Example: formatting a small report

`printf` controls the layout and precision of each output value:

```
awk '!/^#/ && $3 > 12 {
  printf "step=%02d energy=%.2f status=%s\n", $1, $3, $4
}' data/energy.txt
```

```
step=02 energy=15.20 status=WARN
step=04 energy=18.40 status=FAIL
```

Format	Meaning
`%02d`	Integer, at least two digits, padded with zeros.
`%.2f`	Number with two digits after the decimal point.
`%s`	Text.

A newline is explicit with printf

Include `\n` to end the output line. `printf` does not append a newline automatically.

Example: summarising a data file

Accumulate the six energy values, then print one final summary:

```
awk '!/^#/ && NF == 4 {sum += $3; n++}
END {
  if (n > 0)
    printf "count=%d total=%.1f mean=%.2f\n", n, sum, sum/n
  else
    print "No data"
}' data/energy.txt
```

```
count=6 total=73.0 mean=12.17
```

- `sum += $3` adds the energy to the running sum. `n++` increments the count. Uninitialised variables have numeric value zero.
- After the sixth measurement, `sum` is `73.0` and `n` is `6`. Their quotient is the mean.
- `n > 0` prevents division by zero when no records qualify.

For this dataset: `NF == 4` checks the field count. It does not validate whether each value is a valid measurement.

Example: reading and writing separated fields

Read the comma-separated file created with `sed` and keep the step and status of energies above 12:

```
awk -F, '$3 > 12 {print $1, $4}' energy.csv
```

```
2 WARN
4 FAIL
```

Keep the same selection, but separate the output values with a semicolon:

```
awk -F, -v OFS=';' '$3 > 12 {print $1, $4}' energy.csv
```

```
2;WARN
4;FAIL
```

Both commands read the same four fields per record. Only the separator between the two printed values changes.

These simple records have no quoted fields or embedded commas. `-F,` alone is not a general CSV parser.

Example: a report of unusual measurements

Select exceptional statuses, expand one label and format a report:

```
grep -E '[[:space:]](WARN|FAIL)$' data/energy.txt |
  sed 's/WARN/WARNING/' |
  awk '{printf "step=%s energy=%.2f status=%s\n", $1, $3, $4}'
```

```
step=2 energy=15.20 status=WARNING
step=4 energy=18.40 status=FAIL
```

1. `grep` keeps the measurements from steps `2` and `4`.
2. `sed` expands the status in the first selected row from `WARN` to `WARNING`.
3. `awk` adds field labels and prints both energies with two digits after the decimal point.

Inspect the intermediate results

Run the first stage, then the first two stages, then the full pipeline. Add `> alerts.txt` after the final command to save the report.

Example: saving selected data and computing a mean

Keep successful measurements, save their rows and compute their mean energy:

```
grep -E '[[:space:]]OK$' data/energy.txt |
  tee ok.txt |
  awk '{sum += $3; n++}
       END {
         if (n > 0) printf "count=%d mean=%.2f\n", n, sum/n
         else print "No data"
       }'
```

```
count=4 mean=9.85
```

- The selected steps are `0`, `1`, `3` and `5`. Their energies sum to `39.4`.
- After `tee`, `ok.txt` contains the four selected measurements. The final `awk` command receives the same rows.
- The terminal shows their count and mean energy. The saved file contains the measurements used to compute that summary.

Predict: what does `awk` print if no records have status `OK`?